科目：データベースの基礎

第7回 正規化理論：更新時異状と正規化

講師：鈴木源吾

お知らせ

- **小テスト1 レポート**（7点未満・欠席の人）の締切は **今日の講義開始30分後**（満点7点）
  - 締切後も 2/19 18:30 まで提出できる（満点6点）
- **小テスト2 は 11/25（第9回）**．範囲は第4〜7回
  - 持ち込みは第4〜7回のポイント資料のみ
  - 今回の正規化（型J・K）は，第2回の第1正規化（小テスト1 問3）の続き

お知らせ：学習支援センターの補習

- 学習支援センターが **毎週，小テストレポートの補習** を行う
  - **火曜4限**：留学生　／　**水曜3限**：留学生以外
  - 場所は原則 **S301**（11/18 は **S308**）
- **対象**：次の両方に当てはまる人
  - 小テストが **5点未満または欠席**
  - 締切を過ぎた演習の **過半数が未提出**
- **内容**：解答例を見ずに解き直し，質問しながらレポートを仕上げる．未提出の演習もその場で提出する
- **小テスト1**：対象のうちレポート未提出の人は **今日の3限**（留学生は 11/17（火）4限）から
- 演習の未提出が多いと単位が危うくなる．対象の人は必ず参加すること

前回の復習

1. データベースの設計とは？
    - スキーマを決めること．概念設計 → 論理設計 → 物理設計
2. 実体関連モデルの概要
    - 実体型（四角）・関連型（ひし形）・属性（楕円）・主キー（下線）
3. 実体関連モデルの詳細
    - 多重度：相手1つに対するこちら側の個数を，こちら側に 最小..最大 で書く
    - 汎化階層，表記法の違い（情報処理試験・UML）
4. 型I：表から ER 図を描く（外部キーは関連型にする）

第6回演習の解説

- 第6回ポイント資料の裏面（演習1・確認問題）の正解を解説する
- 解説は，演習の締切（講義開始30分後）のあとに行う
- 解答例は電子的には配布しない．**自分の解答に転記すること**（解説中の撮影は不可）
  - 講義中に自分の解答へ書き足す・訂正するのはよい．そのまま試験に持ち込める
- 提出がまだの人は，WebClass の「【締切後】第6回 演習」から提出する（0.7点で評価）

今回のトピック

1. 正規化の背景と概要（更新時異状）
2. 関数従属性と第2正規形
3. 第3正規形
4. 今回の演習と小テストの「型」：**関数従属を見つける・正規形に分解する**

教科書：第7章 p.73〜86（7.1 更新時異状の発生，7.2 情報無損失分解，7.3 関数従属性），第8章 p.87〜91（8.1 第2正規形，8.2 第3正規形）．講義は教科書より易しくした内容

Topic 1

正規化の背景と概要

トピックのキーワード

- 更新時異状（削除・挿入・修正）
- 正規化・正規形

よりよいデータベース設計をするために

- リレーショナルデータベースは，表形式で情報を表現した
- では，どんな表を作ればよいのだろう
- 「良い表」と「悪い表」ってあるの？
- 項目をならべれば終わりじゃないの？

→「正規化」は「良い表」を設計するための指針を教えてくれる

★超重要★：正規化を知ることで，データについての理解が大きく深まり，1ランク上にあがれる

素朴にリレーションを作ると困ること

注文

顧客名	商品名	数量	単価	金額
A商店	テレビ	3	198,000	594,000
Bマート	テレビ	10	198,000	1,980,000
Bマート	洗濯機	5	59,800	299,000
C社	餅つき機	1	29,800	29,800

下線＝主キー {顧客名, 商品名}（2つの属性で1つのキー）

- 例：リレーション「注文」．注文伝票の内容をそのまま並べた
- 主キーは {顧客名, 商品名}（2つの属性で1つのキー）
- 適切に設計しないと，更新するときに良くない状況が起こる．これを**更新時異状**と呼ぶ

更新時異状（1）削除時異状

注文

顧客名	商品名	数量	単価	金額
A商店	テレビ	3	198,000	594,000
Bマート	テレビ	10	198,000	1,980,000
Bマート	洗濯機	5	59,800	299,000
C社	餅つき機	1	~~29,800~~	29,800

注文を取り消す（削除）

→「餅つき機は 29,800円」
　という事実も消える

- C社からの餅つき機の注文が取り消されると，タプルが削除される
- すると「餅つき機が単価 29,800円である」という事実も同時に失われる
- 事実を残すために (null, 餅つき機, null, 29800, null) を入れることもできない
  - 主キー制約により，主キーの属性（顧客名）に空値（null）は許されない

更新時異状（2）挿入時異状

注文

顧客名	商品名	数量	単価	金額
A商店	テレビ	3	198,000	594,000
Bマート	テレビ	10	198,000	1,980,000
Bマート	洗濯機	5	59,800	299,000
C社	餅つき機	1	29,800	29,800

新商品を登録したい → 顧客名（主キーの一部）が空値になるので挿入できない

顧客名	商品名	数量	単価	金額
~~null~~	電子レンジ	null	74,800	null

- 新商品 電子レンジ（74,800円）．まだ注文がないので，顧客名・数量・金額は空値
- 主キー制約によりこのタプルは挿入できない →「電子レンジは 74,800円」を記録できない

更新時異状（3）修正時異状

注文

顧客名	商品名	数量	単価	金額
A商店	テレビ	3	198,000	594,000
Bマート	テレビ	10	198,000	1,980,000
Bマート	洗濯機	5	59,800	299,000
C社	餅つき機	1	29,800	29,800

テレビを 148,000円に値下げ

→ 2つのタプルを同時に修正

1つだけ直すと矛盾する

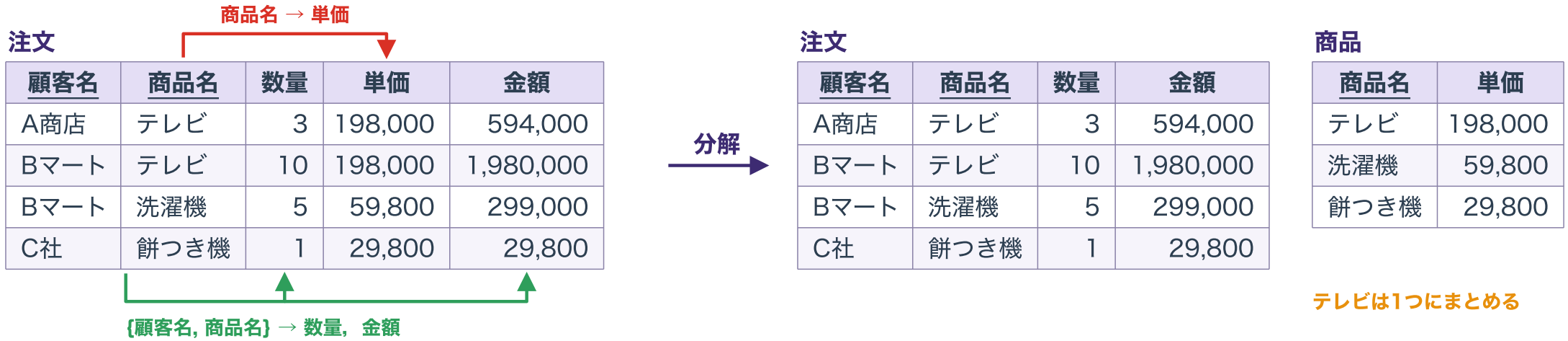
- テレビの単価が 198,000円から 148,000円に値下げになった
- 1番目のタプルだけ修正すると，2番目のタプルと矛盾する
- 現実世界では1つの変更なのに，2つのタプルを修正しなければならない

更新時異状は，なぜ起こるのか？

2つの異なる「もの（実体）」が，1つのリレーションに混在しているから

- 「どの顧客がどの商品を何個注文したか」（注文）
- 「ある商品がいくらするか」（商品）

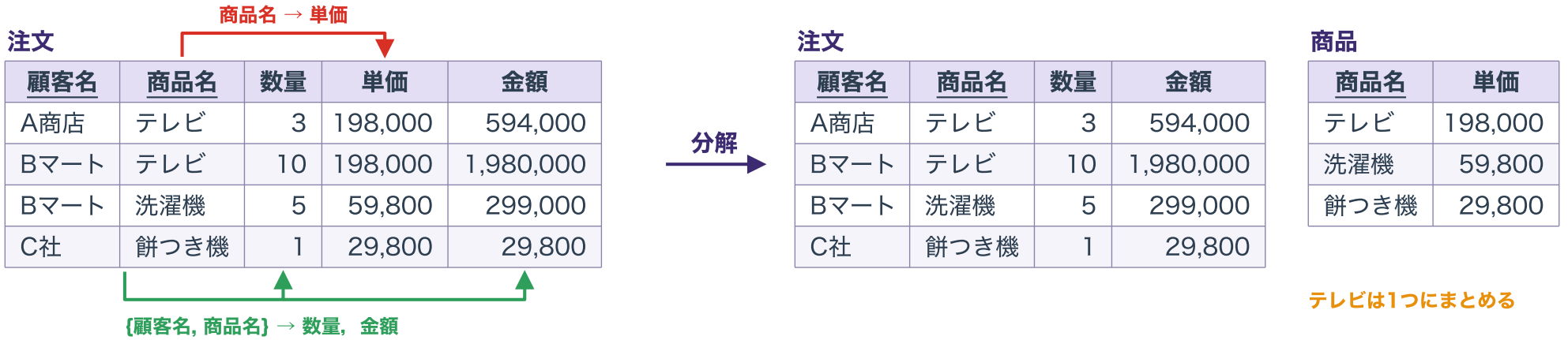
は別に管理すべき．商品名と単価を，注文から分離すればよい



正規化・正規形

- リレーションを，（良い性質と考えられる）一定の制約を満たす形に変形することを**正規化**と呼ぶ
- その制約を満たすリレーションの形を**正規形**と呼ぶ
- 正規形には，満たす条件によっていろいろある．本講義では第3正規形まで．実用上はそれで十分なことが多い
  - 第1正規形（第2回）→ **第2正規形** → **第3正規形** → ボイス・コッド正規形 → 第4正規形 → 第5正規形
  - 後ろの正規形は，前の正規形の条件を含む（第3正規形なら第2正規形でもある）

分解して更新時異状は解消したか？



- 削除：餅つき機の注文を取り消しても，商品の単価は残る
- 挿入：新商品 電子レンジ（74,800円）は，商品に追加すればよい
- 修正：テレビの値下げは，商品の1つのタプルを修正すればよい
  - ただしこの例では，金額（＝数量×単価）も変わる．これは正規化ではなく，計算で求まる値（導出データ）を持つことの問題

分解のルール：元に戻せること（情報無損失分解）

- 分解した2つのリレーションを**共通の属性で結合**すると，元のリレーションに戻る
  - 注文（顧客名, 商品名, 数量, 金額）と 商品（商品名, 単価）は，**商品名**で結合すると元の注文に戻る
- だから，分離した属性を決める属性（商品名）は**両方に残す**
  - 商品名を注文から消すと，どの注文がどの商品か分からなくなる（情報が失われる）
- 同じタプルは1つにまとめる（テレビ 198,000 は1つ）．リレーションは集合なので重複はない
- 教科書では 7.2 情報無損失分解（p.75〜）

Topic 2

関数従属性と第2正規形

トピックのキーワード

- 関数従属性 X → Y
- 部分関数従属・完全関数従属
- 第2正規形

リレーションが1つの「もの」を表すとは？

Topic 1 の例では，

2つの異なる「もの（実体）」が1つのリレーションに混在していた

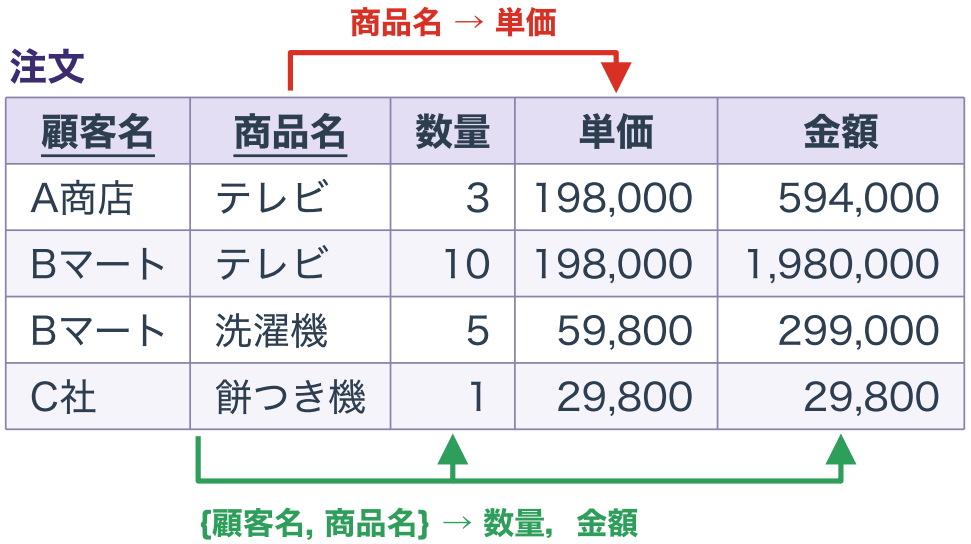
状態を，リレーションを分解することによって，

1つのリレーションが，1つの「もの（実体）」を表すようにした

では，1つの「もの（実体）」であることを，どうやって判断するのだろうか？

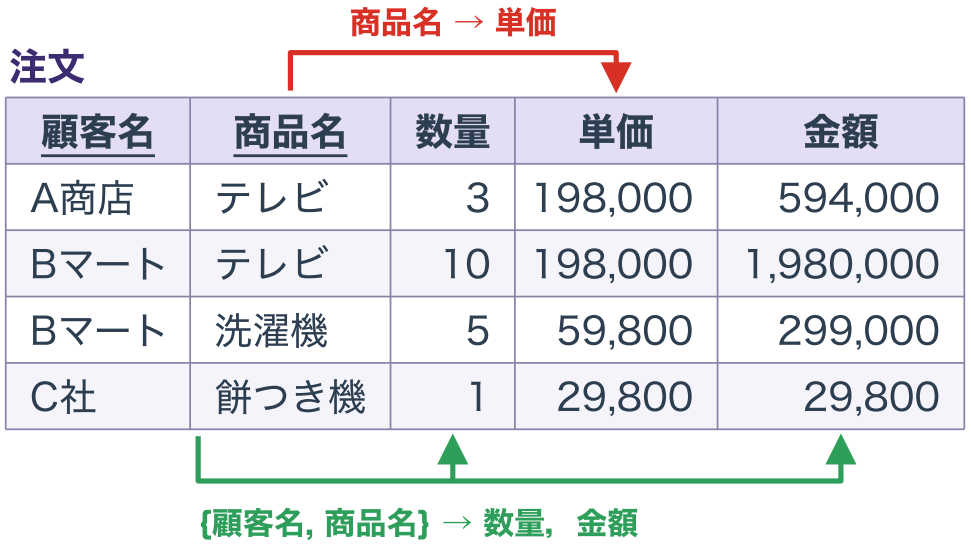
→ これに答えを与えるのが関数従属性という概念である

関数従属性とは？



- 「商品名の値を1つに決めれば，単価は必ず同じ値になる」（テレビ → 198,000 など）
- このような属性間の従属（依存）関係を**関数従属性**と呼ぶ
- 「単価は商品名に**関数従属する**」と言い，**商品名 → 単価** と書く
  - X → Y：「X の値が決まれば，Y の値が1つに決まる」

関数従属性とは？（続き）



- 金額は？ 商品名だけでは決まらず，顧客名と商品名で初めて決まる → **{顧客名, 商品名} → 金額**（数量も同じ）
- 主キーが決まれば，すべての属性が決まる（主キー → 他のすべての属性）
  - 単価も {顧客名, 商品名} → 単価 ではある．しかし，**その一部の商品名だけで決まる**

部分関数従属と完全関数従属

- 単価は，主キー {顧客名, 商品名} の**一部**（商品名）に関数従属している
  - これを**部分関数従属**と呼ぶ．単価は主キーに部分関数従属している
- 数量・金額は，主キーの一部では決まらず，主キー全体で初めて決まる
  - これを**完全関数従属**と呼ぶ（正式な定義は教科書 p.80）
- 部分関数従属があると，「主キーの一部で決まる別のもの（商品）」が混在している

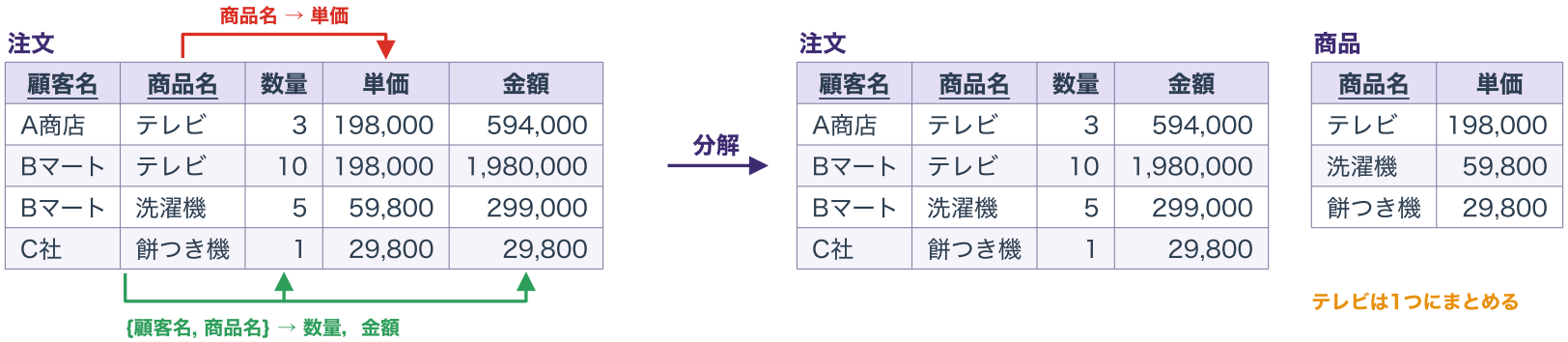
→ 主キーの一部に関数従属している属性があったら，それを別のリレーションに分離する

第2正規形の定義

第2正規形とは，次の2つの条件を満たすことである（教科書 p.88）

- **第1正規形であること**
- **すべての非キー属性が，各候補キーに完全関数従属していること**
  - 非キー属性：どの候補キーにも属していない属性
- 本講義では，主キーで考える：**主キーの一部に関数従属している非キー属性がない**
  - 厳密には候補キーすべてで成り立つ必要がある．主キーに限るほうが理解しやすいので，以降もそう述べる
- 主キーが1つの属性だけなら，「主キーの一部」がないので必ず第2正規形

第2正規形への正規化



1. 主キーの一部に関数従属している非キー属性を見つける（例：単価）
2. その属性と，それを決める主キーの一部（例：商品名）を別リレーションに分離する（例：商品．主キーは商品名）
3. 元のリレーションからは単価だけを消す．**決める側（商品名）は残す**
4. 同じタプルは1つにまとめる

正規化による分解と性能のトレードオフ

- 第2正規化・第3正規化は，リレーションを分解していくプロセスである
- リレーションを分解すると，検索時に結合（ジョイン）する必要が増える
- 結合は計算量が多いから，性能が劣化する
- 正規化と性能はトレードオフにある
- 性能を重視し，あえて正規化しない選択をすることもある
    - その場合，アプリケーション側で，更新時に適切な処理を行う必要がある

Topic 3

第3正規形

トピックのキーワード

- 第2正規形でも起こる更新時異状
- 推移的関数従属
- 第3正規形

第2正規形でも起こる更新時異状

社員

社員番号	社員名	給与	所属	勤務地
0650	山田太郎	50	K55	神奈川
1508	鈴木花子	40	K41	東京
0231	田中桃子	60	K41	東京
2034	佐藤一郎	40	K55	神奈川
2100	森虎丸	40	K58	静岡

主キー：社員番号

前提：部門の所在地は1つ
（K55 は必ず神奈川）

- リレーション「社員」．主キーは社員番号（他に候補キーなし）
- 主キーが1つの属性だけなので，第2正規形（主キーの一部に従属する属性はない）
- 前提：部門（所属）の所在地は1つに決まる．例）K55 の所在地は必ず神奈川

第2正規形でも起こる更新時異状（削除・挿入）

社員

社員番号	社員名	給与	所属	勤務地
0650	山田太郎	50	K55	神奈川
1508	鈴木花子	40	K41	東京
0231	田中桃子	60	K41	東京
2034	佐藤一郎	40	K55	神奈川
2100	森虎丸	40	K58	静岡

2100 森虎丸が退職（削除）

→「K58 は静岡」という
事実も消える

- 削除：2100 森虎丸が退職すると，「部門 K58 が静岡にある」という事実が失われる
- 挿入：新しい部門 K45（千葉）ができても，(null, null, null, K45, 千葉) は主キー（社員番号）が空値なので挿入できない

第2正規形でも起こる更新時異状（修正）

社員

社員番号	社員名	給与	所属	勤務地
0650	山田太郎	50	K55	神奈川
1508	鈴木花子	40	K41	東京
0231	田中桃子	60	K41	東京
2034	佐藤一郎	40	K55	神奈川
2100	森虎丸	40	K58	静岡

K41 が東京から千葉へ移転

→ 2つのタプルを同時に修正

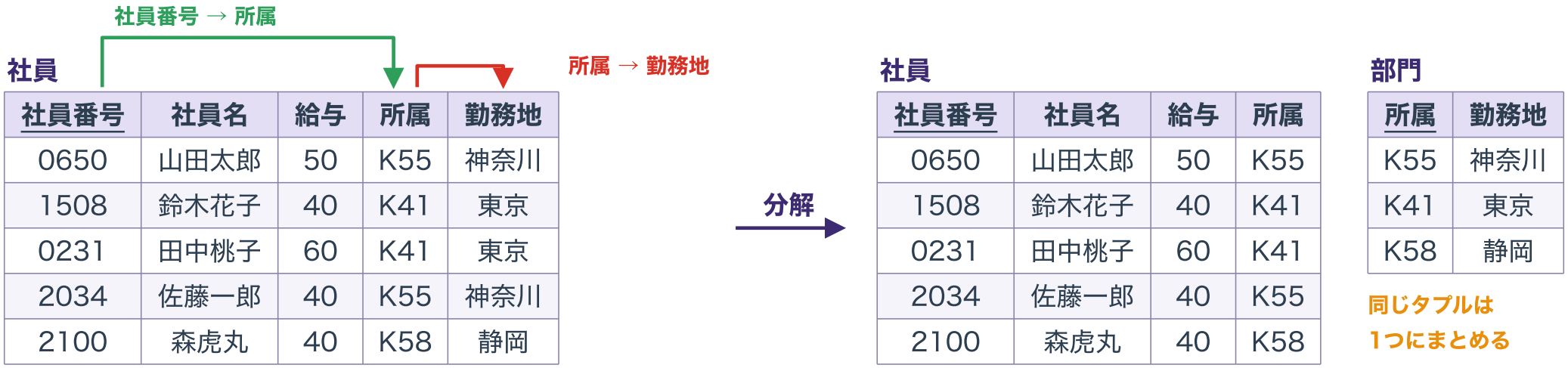
- 部門 K41 が東京から千葉に移転すると，K41 のタプル2つを両方修正しなければならない
- 森虎丸が K55（神奈川）に異動すると，(2100, 森虎丸, 40, K55, 神奈川) に修正する
  - これはよいが，「K58 は静岡」という事実が失われる

第2正規形でも，更新時異状はなぜ起こるのか？

やはり，2つの異なる「もの（実体）」が1つのリレーションに混在するから

- 「**社員**」に関する情報を1つのリレーションにしたが，その中に
- 「**部門**」に関する情報（どの部門がどこにあるか）が隠れている

部門の情報を別に管理できるように，所属と勤務地を分離する



関数従属性でこの例を説明する



社員番号	社員名	給与	所属	勤務地
0650	山田太郎	50	K55	神奈川
1508	鈴木花子	40	K41	東京
0231	田中桃子	60	K41	東京
2034	佐藤一郎	40	K55	神奈川
2100	森虎丸	40	K58	静岡

社員番号 → 所属 → 勤務地

勤務地は社員番号に
推移的に関数従属する

- すべての非キー属性は，主キー 社員番号 が決まれば1つに決まる（社員番号 → 所属，社員番号 → 勤務地）
- しかし勤務地は，非キー属性の**所属だけでも決まってしまう**（所属 → 勤務地）のが問題

推移的関数従属

- A → B，B → C という形で，B を介して A と C が間接的に関数従属することを，**推移的関数従属**と呼ぶ
  - 「C は A に推移的に関数従属する」と言う
  - 例：社員番号 → 所属 → 勤務地．勤務地は社員番号に推移的に関数従属する
  - B（所属）は**非キー属性**．非キー属性が別の非キー属性を決めている

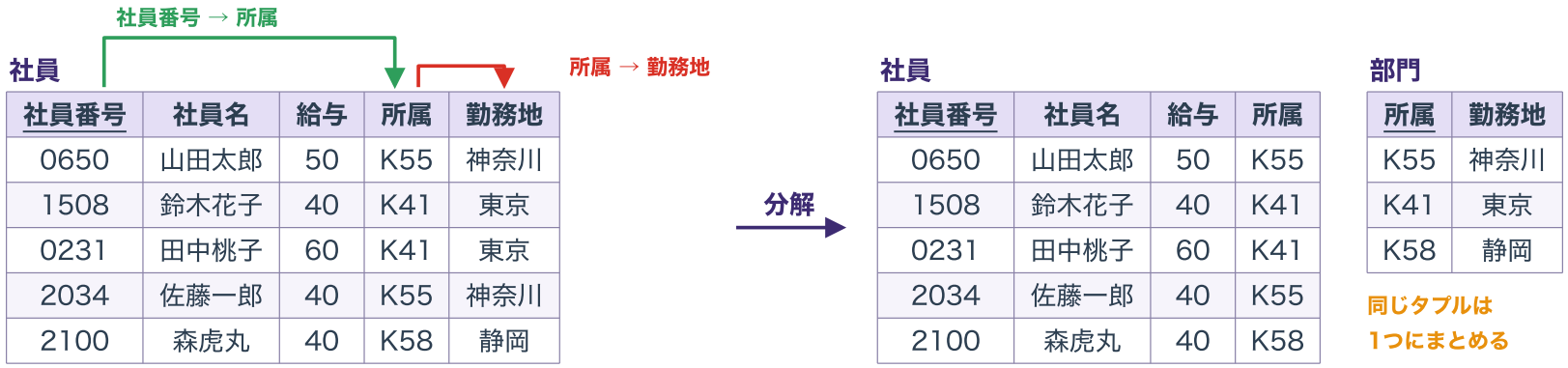
	部分関数従属	**推移的関数従属**
決める側	**主キーの一部**	**非キー属性**
例	商品名 → 単価	所属 → 勤務地
反する条件	第2正規形	第3正規形

第3正規形の定義

第3正規形とは，次の2つの条件を満たすことである（教科書 p.91）

- **第2正規形であること**
- **すべての非キー属性が，どの候補キーにも推移的に関数従属しないこと**
  - 非キー属性：どの候補キーにも属していない属性
- 本講義では：**非キー属性が，別の非キー属性に関数従属していない**

第3正規形への正規化



1. 主キーに推移的に関数従属している非キー属性を見つける（例：勤務地）
2. その属性と，推移の中間の属性（例：所属）を別リレーションに分離する（例：部門．主キーは所属）
3. 元のリレーションからは勤務地だけを消す．**所属は残す**（社員がどの部門か分からなくなる）
4. 同じタプルは1つにまとめる（K41・東京 は1つ）．手順は第2正規化と同じ

Topic 4

今回の演習と小テストの「型」

型J：関数従属を見つける

- 表と前提から，問題になる関数従属を **X → Y** で書き，種類と，反する正規形を答える
- 見分け方：**決める側（X）は何か**
  - X が主キーの一部 → **部分関数従属** → 第2正規形の条件に反する
  - X が非キー属性 → **推移的関数従属** → 第3正規形の条件に反する

例	主キー	関数従属	種類	反する正規形
注文	{顧客名, 商品名}	商品名 → 単価	部分	第2正規形
社員	{社員番号}	所属 → 勤務地	推移的	第3正規形

型K：第2・第3正規形に分解する

- 分解後の各リレーションの**スキーマ（主キーに下線）とタプル**を書く

注文 (<u>顧客名</u>, <u>商品名</u>, 数量, 金額)
　= { (A商店, テレビ, 3, 594000), (Bマート, テレビ, 10, 1980000), (Bマート, 洗濯機, 5, 299000), (C社, 餅つき機, 1, 29800) }
商品 (<u>商品名</u>, 単価)
　= { (テレビ, 198000), (洗濯機, 59800), (餅つき機, 29800) }

- チェック：① 決める側の属性は両方に残っているか　② 同じタプルを1つにまとめたか　③ 主キーに下線があるか

今回の小テストの「型」（小テスト2：11/25）

- **型J：関数従属を見つける**
  - X → Y を書き，部分関数従属／推移的関数従属のどちらか，第何正規形の条件に反するかを答える
- **型K：第2・第3正規形に分解する**
  - スキーマ（主キーに下線）とタプルを書く．同じタプルは1つにまとめる
- 例はポイント資料の表面．小テスト2 の範囲（第4〜7回）の型はF・G・H・I・J・K

第7回演習と次回

- ポイント資料の裏面を，表面の例にならって自分の手で解く
  - 演習1：型Jの類題（演習2の2つのリレーションの関数従属）
  - 演習2：型Kの類題（第2正規化：学部学科，第3正規化：学生）
  - 確認問題：更新時異状の種類，正規形の判定
- 提出：表面・裏面を撮影して WebClass「第7回 演習」へ．締切：**次回（11/18）の講義開始30分後**
- 次回：第8回 データベース管理システム．予習：教科書 第9章 p.100〜114